JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 09 OF 12 - STUDY STEP FW-09

Persistence, SQL, JPA, and Caching

Relational Models, Indexes, Query Plans, Transactions, Hibernate, Redis, and Consistency

BY

Vinay Reddy Kalluri

Reason from relational invariants through indexes, query plans, isolation, locking, connection pools, JPA lifecycle, and cache consistency.

SQL | INDEXES | TRANSACTIONS | JPA | HIBERNATE | REDIS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `FW 08 - Redis`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This book develops the persistence judgment needed to design correct, measurable Java data paths instead of treating databases, ORMs, and caches as interchangeable storage APIs.

Readiness map

Decision	Evidence
START HERE	A query, index, lock, or isolation choice controls correctness or latency; An ORM lifecycle or fetch plan creates hidden work; A connection pool or migration affects deployment safety; A cache creates a second consistency boundary.
PREPARE FIRST	FW-01 MySQL and FW-02 Hibernate/JPA foundations; FW-05 service and REST contracts.
MOVE ON WHEN	Design relational schemas and evidence-based indexes; Reason about isolation, locks, pools, migrations, and outbox delivery; Use JPA and Hibernate lifecycle and fetching deliberately; Design cache keys, invalidation, TTL, and stampede controls.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 08 - Redis	**YOU ARE HERE** **FW 09 - Persistence, SQL, and Caching**	FW 10 - Apache Kafka and Spring Kafka

A note from Vinay

Persistence and caching questions are boundary questions: which store is authoritative, when can copies disagree, and how is recovery observed? Draw the failure window before proposing a pattern.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Relational Modeling, Index Design, and Query Plans

Learning objectives

After this chapter, you should be able to:

- translate business invariants into keys, constraints, and transactionally consistent relational structures;
- distinguish normalization decisions from physical access-path decisions;
- order composite-index columns from query predicates and ordering requirements rather than a slogan;
- explain selectivity, cardinality, covering/index-only access, and write amplification;
- read `EXPLAIN` as evidence about one engine, schema, statistics set, and parameter context;
- choose offset, keyset, or snapshot pagination from product semantics; and
- discuss query performance without presenting a vendor-specific plan as a SQL or JPA guarantee.

WHY IT WORKS | 1. Model facts and invariants before entities

Intuition and formal contract

A relational schema represents facts as tuples and protects some invariants through declared constraints. Begin with identity, cardinality, optionality, and lifecycle:

```text
Customer 1 ---- * Order 1 ---- * OrderLine * ---- 1 Product
                      |
                      +---- state transitions and version
```

Ask:

- What is a stable identity, and who assigns it?
- Can the natural key change? Is a surrogate key still accompanied by a unique business constraint?
- Is the relationship one-to-one, one-to-many, or many-to-many?
- Is absence meaningful, unknown, or invalid?
- Which facts must change atomically?

- Which history must remain after current state changes?
- Which data is tenant-scoped or sensitive?
- Which uniqueness rules apply globally versus within a tenant?

A primary key identifies a row. A unique constraint protects an alternate identity. A foreign key protects referenced existence under its action rules. A check constraint protects a row-level predicate supported by the database. Application validation improves feedback but cannot replace a concurrent database guard.

Concrete schema

SQL

```sql
create table customer_order (
    id bigint generated by default as identity primary key,
    tenant_id bigint not null,
    external_order_id varchar(80) not null,
    customer_id bigint not null,
    status varchar(24) not null,
    total_cents bigint not null check (total_cents >= 0),
    version bigint not null,
    created_at timestamp with time zone not null,
    updated_at timestamp with time zone not null,
    constraint customer_order_business_key
        unique (tenant_id, external_order_id)
);

create table order_line (
    order_id bigint not null references customer_order(id),
    line_number integer not null check (line_number > 0),
    product_id bigint not null,
    quantity integer not null check (quantity > 0),
    unit_price_cents bigint not null check (unit_price_cents >= 0),
    primary key (order_id, line_number)
);
```

`external_order_id` alone is not globally unique because identity is tenant-scoped. A composite primary key for line numbers expresses ownership and prevents duplicates within one order. If product price changes later, preserving `unit_price_cents` on the line records the commercial fact at order time rather than deriving history from the current product row.

Normalization and deliberate duplication

Normalization reduces update anomalies by storing a fact once at the relation where it belongs. It is not a commandment to eliminate every repeated byte. An order line's purchase price and a product's current price are different facts. An immutable event payload can duplicate selected state because it has different lifecycle and replay needs.

Denormalize only with a maintenance contract:

TEXT

```text
source of truth + update mechanism + acceptable lag
+ repair/rebuild procedure + consistency monitoring
```

A cached aggregate column such as `order_total_cents` can be correct if the same transaction updates lines and total, a database mechanism enforces it, or the value is explicitly eventual and reconcilable. "Faster reads" without an update/repair contract creates silent divergence.

Modeling traps

- a nullable foreign key used to encode five unrelated states;
- soft deletion without deciding whether unique keys may be reused;
- status stored as arbitrary text with no application/database validation;
- tenant ID omitted from a unique constraint or access predicate;
- money stored as binary floating point;
- timestamps without an instant/time-zone contract;
- comma-separated IDs replacing a join table;
- JSON used to hide a stable relational structure that needs constraints and queries;
- cascade delete selected without considering audit, retention, and accidental blast radius.

2. Indexes are ordered access paths

Core model

For a common B-tree-style index, the indexed keys are maintained in an order. A composite index on `(tenant_id, status, created_at DESC, id DESC)` can support a contiguous key range beginning with equality on `tenant_id` and `status`, then an ordered range/traversal on the remaining keys.

SQL

```
select id, status, total_cents, created_at
from customer_order
where tenant_id = :tenant
  and status = :status
  and (created_at, id) < (:cursor_time, :cursor_id)
order by created_at desc, id desc
limit :limit;
```

Candidate index:

SQL

```
create index order_tenant_status_created_id
    on customer_order
        (tenant_id, status, created_at desc, id desc);
```

Column order follows the query family, not "most selective first" as a universal rule. Equality predicates often form a prefix; the first range/order component affects how much of the remaining key order is usable. Different

databases support included columns, partial/filtered indexes, expression indexes, descending keys, skip scans, and index intersection differently.

Selectivity and cardinality

Cardinality is the number of distinct values (or estimated distinct values) in a column or expression. **Selectivity** describes how much a predicate narrows the table. A boolean `active` column often has low cardinality; an `id` has high cardinality. But skew matters: `status='FAILED'` might select 0.01% while `status='COMPLETED'` selects 90%.

Optimizers estimate row counts using statistics. Correlated columns can defeat independent assumptions: country and postal code are not independent. Parameter values can produce different ideal plans. Stale or low-detail statistics can cause a poor join order or access path even when an index exists.

Recognition rule: ask "how many rows enter and leave each operator?" rather than merely "was an index used?" A sequential scan can be optimal for a large fraction of a small or cached table. An index lookup can be bad when it causes many random table fetches.

Covering and index-only access

An index *covers* a query when it contains all values needed by the predicates and projection, allowing the engine in favorable conditions to avoid or reduce table/heap access. This is not a portable guarantee. Visibility rules, storage engine structure, page state, and selected columns affect whether an index-only plan is possible.

Adding projection columns to an index can:

- reduce reads for a critical query;
- enlarge the index and memory footprint;
- increase write, WAL/log, replication, and maintenance work;
- reduce cache efficiency;
- overlap other indexes.

The decision rule is workload evidence: optimize a measured important query family and account for write cost. Do not create one index per observed query without consolidating prefixes and measuring.

Composite-index interview method

For each candidate query:

1. write exact predicates, order, projection, and limit;
2. estimate tenant/data distribution and result cardinality;
3. place stable equality prefix columns that define the searched range;
4. place range and order columns so the desired traversal is possible;
5. add a unique tie-breaker for total order;
6. consider covering columns only for important read paths;
7. test writes and competing queries;
8. inspect the plan with production-like data and parameters.

Common mistakes

- assuming `(a,b)` supports every query on `b` efficiently;
- adding separate single-column indexes when a composite query needs an ordered prefix;
- forgetting that foreign-key columns may need indexes for joins/deletes depending on engine/workload;
- wrapping an indexed column in a function without an expression-index strategy;
- comparing different types or collations through implicit conversion;
- selecting every entity column and defeating a covering read model;
- leaving redundant prefix indexes without measuring whether another workload needs them;
- ignoring insert hotspots caused by monotonically clustered keys in a distributed/partitioned store.

3. Reading EXPLAIN as experimental evidence

Plan model

A query plan is a tree or graph of physical operators. Typical concepts include scans, lookups, filters, sorts, joins, aggregation, materialization, and parallel exchange. The optimizer chooses a plan using estimates and a cost model. `EXPLAIN` terminology and cost units are vendor-specific.

Inspect:

- estimated versus actual rows at each node;
- loops: per-loop values can hide multiplied work;
- scan predicate versus residual filter;
- join algorithm and build/probe or outer/inner side;
- explicit sort and whether it spills;
- buffers/pages/physical I/O if available;
- execution versus planning time;
- memory and temporary-space use;
- parallel workers planned/launched;
- partitions pruned;
- locks and side effects of an analyze/execution option.

A first large estimate error often propagates upward and causes a bad join or memory decision. Fixing statistics, query shape, parameterization, or data model can be more robust than forcing a plan.

Example reasoning

Suppose a plan reports:

TEXT

```
Limit
  -> Sort by created_at DESC, id DESC
       -> Scan customer_order
            Filter tenant_id = ? and status = ?
```

For a large tenant, the sort and scanned rows may dominate. The candidate composite index aligns with filter and ordering. After adding it, verify that the plan traverses the desired key range and that actual rows are close to estimates. Then measure write overhead and index size. Do not declare success from the word `Index`; compare latency distribution, logical/physical reads, and affected writes.

Parameter and environment traps

Plans depend on:

- engine and exact version/configuration;
- schema, indexes, constraints, and statistics;
- table size, distribution, correlation, and freshness;
- parameter values and prepared-statement plan policy;
- cache warmth and concurrent load;
- transaction visibility and isolation;
- projection width and client fetch behavior.

Use sanitized production-like distributions, not ten development rows. An execution-enabled plan can run mutations; place it inside a safe rollback/testing process only when the database documentation confirms behavior, and never experiment casually on production.

4. Offset, keyset, and snapshot pagination

Formal semantics

Offset pagination selects an ordered relation and discards the first k rows. Work can grow with k, and concurrent changes can move row positions:

SQL

```
select ...
from customer_order
where tenant_id = :tenant
order by created_at desc, id desc
limit :limit offset :offset;
```

Keyset pagination continues after an explicit key tuple:

SQL

```
where tenant_id = :tenant
  and (created_at, id) < (:last_time, :last_id)
order by created_at desc, id desc
limit :limit_plus_one;
```

The order must be total. `created_at` alone can tie; `id` breaks ties. The comparison direction must match each sort direction. For mixed directions, derive the lexicographic predicate carefully rather than copying tuple syntax blindly.

A snapshot/export approach fixes a read timestamp, materialized ID set, or generated artifact. It costs storage/work but provides a stable exhaustive traversal. Keyset over live data is not automatically a snapshot.

Decision matrix

Requirement	Offset	Keyset	Snapshot/export
jump to arbitrary page number	strong	weak	depends on artifact index
predictable deep-page work	weak	strong with aligned index	strong after generation
simple UI for small results	strong	acceptable	excessive
stable exhaustive export during writes	weak	partial/live semantics	strongest explicit contract
exact total count	separate query	separate query	can record generated total

Counting can be more expensive than fetching one page. "Total" must specify the same filters, authorization boundary, and snapshot semantics. Offer an approximate total only if labeled and useful.

Failure walkthrough

Orders A, B, C are sorted newest to oldest. The client reads offset page `[A,B]`. A new order X is inserted at the front. Offset 2 now returns `[B,C]`; B duplicates. With keyset after B's tuple, X is outside the continuation range and C follows. If B's sort key later changes, the live traversal still needs documented behavior.

TRY IT | 5. Interview questions and model checkpoints

Q1. How do you choose composite-index order?

Model checkpoint: start from exact query families, equality/range predicates, ordering, limits, distribution, and projection. Form a useful searchable prefix and verify with the target engine. "Most selective first" is not a universal algorithm.

Q2. Why might a database ignore an index?

Model checkpoint: a scan may be cheaper; estimates or statistics may differ; the predicate may not match the prefix/expression/type; result fraction or table size may be large; parameter plan policy or visibility/table-fetch cost may dominate. Inspect plan evidence.

Q3. What does a covering index cost?

Model checkpoint: space, write amplification, logging/replication, maintenance, and cache footprint. Even when all columns are present, engine visibility/storage rules decide whether table access is avoided.

Q4. Does keyset pagination prevent all duplicates?

Model checkpoint: it avoids position drift from rows inserted before the cursor when order keys are stable. Updates to ordering keys, snapshot changes, cursor misuse, or non-total order can still skip/duplicate.

SDE-2 follow-ups

1. Design indexes for a multi-tenant order dashboard with status filters and two sort modes. Decide whether to support every combination.
2. An index reduced p50 but worsened write p99. List measurement and rollback steps.
3. Estimated rows are 10 but actual rows are 2 million. Trace how this can change join and memory choices.
4. Design a zero-downtime transition from offset links to opaque keyset cursors.

TRY IT | 6. Exercises

1. Normalize a shopping-cart JSON document into relations and list every business constraint that SQL alone can and cannot enforce.
2. Given queries by `(tenant,status,createdAt)` and `(tenant,customer,createdAt)`, propose an index set and identify redundancy.
3. Annotate an actual plan from your target database: estimates, actual rows, loops, I/O, sort, and first divergence.
4. Derive forward and backward keyset predicates for `(score DESC, createdAt ASC, id ASC)`.
5. Explain how a soft-delete predicate changes uniqueness and index design in two database engines.

RECAP | 7. Summary checklist

- [ ] Identity, tenant scope, cardinality, optionality, and history are explicit.
- [ ] Database constraints protect concurrent invariants.
- [ ] Denormalized facts have update and repair contracts.
- [ ] Indexes derive from query families, ordering, and data distribution.
- [ ] Covering benefits are balanced against write and space cost.
- [ ] Plans are read through estimates, actuals, loops, I/O, and spills.
- [ ] Pagination has a total order and explicit live/snapshot semantics.
- [ ] Every performance claim names the database, version, data, parameters, and workload.

8. Query-design laboratory: joins, aggregation, and plan stability

Join reasoning

For each join, identify cardinality before physical algorithm:

TEXT

```
Order (1) -> OrderLine (many)
OrderLine (many) -> Product (1)
```

Joining orders to lines multiplies rows. `count(*)` after that join counts lines, not orders. `count(distinct order.id)` restores parent count at extra work, but a semijoin/`exists` may express the actual question better:

SQL

```
select o.id, o.created_at, o.status
from customer_order o
where o.tenant_id = :tenant
  and exists (
      select 1
      from order_line l
      where l.order_id = o.id
        and l.product_id = :product)
order by o.created_at desc, o.id desc
limit :limit;
```

An index on `order_line(product_id, order_id)` may support finding orders by product; `(order_id, product_id)` supports checking products for a known order. One ordering can serve one direction better. The optimizer can choose nested loop, hash join, merge join, or another engine-specific plan based on estimates and ordering. Do not force an algorithm before measuring.

Aggregation and window decisions

Suppose the API needs each customer's most recent order. A correlated subquery, grouped max joined back, database-specific distinct construct, or window function can express it. A portable window shape is:

SQL

```
select id, customer_id, created_at, status
from (
    select o.*,
           row_number() over (
               partition by tenant_id, customer_id
               order by created_at desc, id desc) as rn
    from customer_order o
    where tenant_id = :tenant
) ranked
where rn = 1;
```

This may sort/process many tenant rows. An aligned index and engine can optimize some paths, but inspect the plan. If the use case is latency-critical and read-heavy, a maintained `latest_order` read model may be justified-with an update/rebuild contract-rather than repeatedly ranking millions of rows.

Sargability and type boundaries

A predicate is useful to an access path only when the engine can map it to its index/search capabilities. Compare:

SQL

```
where created_at >= :start and created_at < :end
```

with:

```sql
where date(created_at) = :day
```

The latter may require evaluating a function on rows unless an expression index/optimizer rewrite applies. Half-open ranges also make timezone boundaries explicit. Bind a timestamp type matching the column and compute the day in the intended business zone before converting to instants.

Leading-wildcard text search, case folding, collation, JSON paths, and geospatial queries often need specialized indexes. A B-tree is not a universal search engine. Select technology from query semantics and measured scale.

Plan-regression checklist

When p99 regresses after a deploy:

1. confirm the exact normalized query and parameter class changed or did not;
2. compare application rows fetched, ORM query count, and DB execution time;
3. capture plans with representative parameters and safe actual metrics;
4. inspect estimates at the first divergence, not only the root duration;
5. check statistics, table/index growth, skew, schema migration, and configuration;
6. check lock/I/O/cache/concurrency evidence;
7. test the smallest reversible change;
8. verify other query/write paths and tail latency;
9. record the plan/workload evidence in a regression test or performance runbook.

Parameterized queries can have different optimal plans for "tiny tenant" versus "whale tenant." Do not solve that with unsafe SQL concatenation. Use supported plan/statistics/query-shape techniques of the selected database and keep values parameterized.

Laboratory checkpoint

An SDE-2 explanation states relational cardinality, estimated/actual rows, access path, join multiplication, sorting/aggregation memory, output width, and application fetch count. Index presence alone is not a performance analysis.

Primary references

- PostgreSQL Documentation, "Indexes": https://www.postgresql.org/docs/current/indexes.html
- PostgreSQL Documentation, "Using EXPLAIN": https://www.postgresql.org/docs/current/using-explain.html
- PostgreSQL Documentation, `SELECT`: https://www.postgresql.org/docs/current/sql-select.html
- Jakarta Persistence specification: https://jakarta.ee/specifications/persistence/

> **Version boundary:** SQL examples are illustrative and closest to PostgreSQL syntax, not portable guarantees. Descending indexes, included columns, row-value comparison, identity syntax, plan fields, and optimizer behavior vary. Java integration examples in this module target Java 21; later JDK features are not required.

SDE-2 CORE CHAPTER

Chapter 2 - Transactions, Isolation, Locks, Pools, Migrations, and Outbox Boundaries

Learning objectives

After this chapter, you should be able to:

- define a transaction from the business invariant and resource boundary;
- distinguish dirty read, nonrepeatable read, phantom, lost update, write skew, and serialization failure;
- choose atomic SQL, optimistic versioning, pessimistic locking, or serializable retry from contention and invariant shape;
- prevent, diagnose, and safely retry selected deadlock/serialization victims;
- size a connection pool as a concurrency limit rather than a throughput multiplier;
- evolve schemas with expand/migrate/contract compatibility; and
- use an outbox and idempotent consumer to cross database/message boundaries without claiming impossible local atomicity.

WHY IT WORKS | 1. Transaction as a recoverable invariant boundary

A transaction groups operations in one resource manager under atomic commit/rollback and isolation behavior. The useful question is not "which methods have an annotation?" It is:

> Which state changes must become visible together so every committed state satisfies the business invariant?

For a bank transfer within one database:

```text
debit source + credit destination + append ledger entries = one local transaction
```

For "update database and call email provider," there is no single local transaction. Email must be derived from recoverable state and retried/deduplicated. Holding the database transaction open during the call increases lock and connection time but does not make the remote side atomic.

JDBC execution skeleton

JAVA

```java
// Dependency-requiring only for the configured JDBC driver/DataSource.
public TransferResult transfer(DataSource dataSource, Command command)
        throws SQLException {
    try (Connection connection = dataSource.getConnection()) {
        boolean originalAutoCommit = connection.getAutoCommit();
        try {
            connection.setAutoCommit(false);
            TransferResult result = executeTransfer(connection, command);
            connection.commit();
            return result;
        } catch (Throwable failure) {
            try {
                connection.rollback();
            } catch (SQLException rollbackFailure) {
                failure.addSuppressed(rollbackFailure);
            }
            throw failure;
        } finally {
            connection.setAutoCommit(originalAutoCommit);
        }
    }
}
```

Production pool integrations normally reset documented session state, but code should not rely on an undocumented cleanup miracle. Use try-with-resources. Do not leak result sets/statements. Map classified SQL failures through vendor error codes/states rather than parsing localized messages. Framework transaction abstractions can handle boilerplate; the invariant and failure semantics remain yours.

2. Isolation anomalies as executions

Phenomena

Use schedules rather than labels:

- **dirty read:** T2 observes T1's uncommitted write; T1 later rolls back;
- **nonrepeatable read:** T1 reads row R, T2 commits a change, T1 reads R again and sees a different value;
- **phantom:** T1 repeats a predicate query and sees a changed set due to concurrent insert/delete/update;
- **lost update:** two transactions derive writes from the same old value and one overwrites the other;
- **write skew:** transactions read a shared condition and update different rows, jointly violating a cross-row invariant;
- **serialization failure:** the database rejects an execution that cannot be ordered under its serializable model; the application may retry the *whole transaction*.

SQL isolation names are not complete portable behavior specifications. Lock-based and MVCC engines map levels differently, and vendor documentation defines actual guarantees. State the engine/version when claiming which anomaly is prevented.

Write-skew walkthrough

Invariant: at least one doctor must remain on call.

```text
T1 reads Alice=on, Bob=on
T2 reads Alice=on, Bob=on
T1 updates Alice=off
T2 updates Bob=off
both commit -> invariant violated
```

Each row update touched a different row, so row-level optimistic versions may not conflict. Solutions include serializable execution with retry, locking a shared invariant row/range, redesigning the data so one atomic constraint can protect it, or a database-specific constraint/procedure. "Use optimistic locking" is incomplete unless its conflict set covers the invariant.

3. Concurrency-control decision table

Pattern	Best fit	Contract	Costs/traps
atomic conditional update	simple numeric/state predicate	success is update count > 0	limited to expressible predicate; translate zero carefully
optimistic version	low/moderate conflicting writes, user edits	`UPDATE ... WHERE id=? AND version=?`	retry or surface conflict; detached stale data; does not cover other rows automatically
pessimistic row/range lock	high-cost conflicts that must be serialized	acquire documented lock before decision	waits, deadlocks, pool occupancy; range behavior engine-specific
serializable transaction	complex invariant requiring serial execution	DB either produces serializable effect or aborts	retries and contention; no remote side effects inside retryable body
application mutex	process-local state only or coordinated external lock with protocol	one owner under defined lease/fencing	single-JVM locks do not protect multiple instances; leases need fencing

Atomic inventory example

```sql
update inventory
set available = available - :requested,
    version = version + 1
where sku = :sku
  and available >= :requested;
```

The statement makes check and decrement one database operation. Update count zero needs a product decision: unknown SKU, insufficient stock, concurrent change, or authorization filtering. Avoid an extra pre-read as the correctness guard; it can improve diagnostics but the conditional update is authoritative.

Optimistic locking example

SQL

```
update customer_order
set note = :note,
    version = version + 1,
    updated_at = :now
where id = :id
  and tenant_id = :tenant
  and version = :expected_version;
```

Zero rows means the precondition failed or the row is invisible/missing. Do not automatically retry a user edit using new state: that can overwrite someone else's intent. For an internally derived commutative transition, a bounded reload/recompute retry may be correct.

4. Locks and deadlocks

A lock protects a resource under an engine-defined compatibility matrix and duration. Resources can be rows, keys, key ranges, pages, tables, metadata, advisory identities, or internal structures. "Row lock" is often an approximation; inspect the target engine.

A deadlock is a cycle in the wait-for graph:

TEXT

```
T1 holds A, waits for B
T2 holds B, waits for A
```

The database detects or times out a victim. Deadlocks are not eliminated by higher timeouts.

Prevention and reduction

- access shared resources in a deterministic order;
- keep transactions short and exclude remote/user think time;
- use indexes so mutations lock/find only intended rows/ranges;
- avoid updating unrelated aggregates inside hot transactions;
- choose isolation and locking that match the invariant;
- bound batch size;
- monitor lock waits and capture deadlock diagnostics;
- retry only the whole transaction, only for classified retryable victims, with jitter and a deadline.

Failure walkthrough: retrying at the wrong level

A transfer debits account A, sends a notification, then updates B. The database aborts as a deadlock victim. Retrying only the B update leaves an unbalanced transfer; retrying the whole block sends the notification twice. Correct design keeps remote notification out of the retryable transaction and records an outbox event atomically. The whole local transaction is retried from a clean connection/state with an idempotent command identity.

5. Connection pools and backpressure

A database connection is a scarce session and concurrency slot. If a service has `N` instances each with pool maximum `P`, potential sessions approximate `N * P` plus jobs, consoles, failover overlap, and other services. The database must support the total with headroom.

Little's Law provides a useful check: average in-flight database work `L = arrival rate λ * average time W`. Increasing the pool above the database's useful concurrency can increase queueing, context switching, locks, and tail latency instead of throughput.

Pool decision inputs:

- measured database capacity and latency under representative queries;
- service instance count and autoscaling maximum;
- request mix and fraction that needs a connection;
- transaction duration distribution, not only average;
- scheduled/background consumers;
- failover and rolling-deployment overlap;
- acquisition timeout shorter than the request deadline;
- leak detection and active/idle/wait metrics.

Do not hold a connection while waiting on HTTP, sleeping for retry, rendering a large response, or processing CPU-heavy work that could happen before/after the transaction. An unbounded application queue merely hides pool saturation until deadlines expire.

Pool failure signatures

- rising acquisition wait with pool at max: database work or capacity bottleneck;
- active below max but requests slow: bottleneck elsewhere or pool configuration/traffic shape;
- many idle connections across many instances: wasted DB session capacity;
- frequent validation failures: network/database lifecycle mismatch;
- connections returned with open transaction/session changes: ownership bug or reset-policy mismatch;
- leak warnings: inspect stack evidence, but long legitimate transactions can resemble leaks.

6. Schema migrations with mixed-version compatibility

During a rolling deployment, old and new application versions may run against one schema. Use expand/migrate/contract:

1. **expand:** add backward-compatible structures-nullable column, new table, index built with the engine's safe procedure;
2. **deploy compatibility:** new code can read old/new form and writes data needed by both when necessary;
3. **backfill/migrate:** bounded, resumable, observable batches; throttle and verify;
4. **switch reads:** use the new representation after correctness evidence;
5. **stop old writes:** deploy and verify;
6. **contract:** later remove old column/constraint/path after rollback window.

Adding a `NOT NULL` column with a computed default, rebuilding a large table/index, or validating a constraint can lock or rewrite depending on engine/version. Check official database documentation and test on production-scale copies. Migration tools order and record scripts; they do not make every DDL operation online or reversible.

Migration mistakes

- destructive rename/drop in the same deploy that changes code;
- one enormous backfill transaction;
- dual write without reconciliation or authoritative source;
- assuming rollback can restore discarded data;
- allowing every instance to race migration on startup without a supported lock protocol;
- ORM auto-DDL in production hiding review and rollout semantics;
- adding an index without measuring replication/log and write impact.

7. Transactional outbox

Contract and flow

When a use case updates domain state and needs to publish an event:

```text
local transaction:
  update domain rows
  insert outbox(event_id, aggregate_id, type, payload, occurred_at)
commit

relay:
  claim/read unpublished events
  publish
  record progress

consumer:
  deduplicate event_id
  apply local transition atomically with inbox marker where appropriate
```

The domain change and intent-to-publish are atomic because both are database rows. Publish and mark cannot generally be atomic across DB and broker, so duplicates remain possible. Consumers must be idempotent. Ordering is only as strong as the outbox claim/publish and broker key/partition protocol; global order is costly and rarely needed.

Relay decisions

- polling versus database-log/change-data-capture approach;
- claim strategy for multiple relay workers;
- batch size and transaction duration;
- retry/backoff and poison-event quarantine;
- per-aggregate ordering key;
- schema/event-version compatibility;
- payload privacy and retention;
- backlog age SLO and reconciliation.

Do not delete an outbox row immediately without operational evidence. Archival/retention depends on replay, audit, privacy, and storage needs. A DLQ does not solve correctness; it stores work requiring a triage/replay protocol.

TRY IT | 8. Interview questions and model checkpoints

Q1. What isolation level prevents lost updates?

Model checkpoint: do not answer from name alone. Define the schedule, database engine behavior, statement shape, and whether atomic predicates/version checks are used. A version predicate explicitly detects overwrite regardless of many read-level subtleties.

Q2. Should the pool maximum equal request threads?

Model checkpoint: no. Size from database useful concurrency, instance count, query time, traffic mix, and deadline. More connections can worsen saturation; admission must be bounded.

Q3. Does outbox give exactly-once delivery?

Model checkpoint: it atomically stores domain change plus publication intent. Relay publish/mark can duplicate; consumers need idempotency/deduplication. Define ordering and retry separately.

Q4. How do you deploy a new non-null field?

Model checkpoint: expand with compatible nullable/default structure, deploy compatible writers/readers, backfill and verify, switch/enforce, then contract later. Exact online DDL depends on the engine/version.

SDE-2 follow-ups

1. A deadlock rate rises after adding an index. Explain how changed plans/order can alter locks and how you would gather evidence.
2. Autoscaling doubled instances and caused pool timeouts. Build a connection-budget calculation.
3. A consumer processed an event, crashed before committing its dedupe marker, and ran twice. Redesign the local atomic boundary.
4. Plan a migration from integer to string/UUID external IDs with two application versions active.

TRY IT | 9. Exercises

1. Draw schedules for lost update and write skew; propose three fixes and state contention tradeoffs.
2. Implement a JDBC optimistic update that distinguishes conflict from database outage without message parsing.
3. Given 40 instances, 20 background workers, and a 600-connection DB budget, propose pool/admission limits and deployment headroom.
4. Design a resumable backfill with checkpoints, throttling, validation, and rollback policy.
5. Specify outbox and inbox tables plus relay/consumer state transitions for per-order event ordering.

RECAP | 10. Summary checklist

- [ ] The transaction matches one local invariant and resource manager.
- [ ] Isolation claims name an execution and target database behavior.
- [ ] Atomic/version/lock/serializable choice matches contention and invariant scope.
- [ ] Deadlocks retry the whole safe local transaction under a deadline.
- [ ] Pool size is part of a system-wide connection and concurrency budget.
- [ ] Migrations tolerate mixed application versions and have a repair path.
- [ ] Outbox publication and consumers are explicitly duplicate-safe.
- [ ] Remote calls and unbounded work are outside database transactions.

11. Transaction incident laboratory

Incident A: "successful" request rolled back

A service catches a duplicate-key exception, logs it, and returns an existing object. At method exit Spring reports an unexpected rollback. The database exception marked the transaction rollback-only; catching it did not restore a valid transactional context.

Evidence and repair:

1. capture the first database exception and transaction state, not only commit failure;
2. identify whether duplicate is an expected race protected by a unique constraint;
3. restructure reserve-or-read so the conflicting insert occurs in a boundary where rollback is expected, or use database-supported atomic upsert/insert-if-absent semantics;
4. do not continue arbitrary ORM work after a failed flush;
5. integration-test two concurrent attempts and assert exactly one durable row/result;
6. classify only the known constraint.

Starting `REQUIRES_NEW` around random repository calls can mask the symptom while breaking the use-case invariant and consuming extra pool connections.

Incident B: deadlocks after batch feature

Two workers update orders from different input order:

```text
worker A locks 0-1 then 0-2
worker B locks 0-2 then 0-1
```

The database chooses a victim. Remediation is not "disable transactions." Sort resource identities and acquire/update in deterministic order, reduce batch size and transaction duration, verify indexes, and retry the whole idempotent local unit for the classified victim. Measure whether a different execution plan still locks in another order. Capture deadlock graph/details supplied by the engine.

Incident C: pool starvation with nested transactions

Sixty request threads each hold one outer transaction connection and invoke audit logic that requires an independent connection. Pool maximum is sixty. Every thread waits for a second connection that cannot become free until outer work completes: a resource deadlock outside the database's lock detector.

Fix semantics first: should audit commit independently if business work rolls back? An outbox or after-commit audit may be more honest. If independent nested transactions are required, pool/concurrency policy must account for peak simultaneous connections and leave headroom-but limiting inbound concurrency is safer than hoping every request never nests.

Incident D: migration causes write outage

A migration adds an index/constraint using a blocking procedure on a large hot table. Application pods are healthy but writes queue behind a schema lock.

Runbook:

- stop/abort the migration only through documented safe database commands;
- reduce user impact with admission/read-only behavior if applicable;
- inspect blocker/wait evidence and replication/log impact;
- redesign as expand plus online/concurrent build or validated-in-stages technique supported by that engine/version;
- test production-scale duration/locks;
- set migration statement/lock timeouts where supported;
- separate deploy and contract cleanup.

Transaction laboratory checkpoint

For every incident, draw connections, transactions, locks, waits, external effects, and commit points. The final answer must say what durable state exists after each exception and which operation identity makes a retry safe.

12. Outbox ordering and retention drill

Suppose two transactions for order 0 - 7 commit events with aggregate sequence 8 and 9. Multiple relay workers can claim them. Publishing 9 before 8 violates a consumer that expects state-machine order even though both rows are durable.

Possible contracts:

- claim/publish one aggregate serially and key Kafka by aggregate ID;
- let broker partition order reflect outbox sequence, ensuring producer submission for one key is ordered under documented client behavior;
- consumers accept only next expected version and buffer/retry gaps;
- consumers treat events as state facts with monotonic source version and ignore stale versions;
- redesign events to be commutative where business meaning permits.

Global commit order across every aggregate is rarely required and can destroy parallelism. State per-aggregate versus partition versus global order explicitly. Database auto-increment IDs are not automatically business commit order under concurrent transactions; allocation and commit can differ.

Retention connects publisher and consumer recovery. Deleting an outbox row after publish may be fine for relay operation if Kafka retention/replay and domain records satisfy audit; it may be wrong when the outbox is the only legal record. Inbox/deduplication retention must exceed the plausible redelivery/replay window. If an operator replays events older than the inbox TTL, effects can repeat. A replay tool should use a new controlled operation identity or restore dedupe evidence under an approved protocol.

Monitor:

- oldest unpublished age and count;
- per-aggregate sequence gaps;
- claim lease expiry/steal;
- publish attempt and classified failure;
- published-but-unmarked duplicate rate;
- poison-event quarantine age;
- inbox duplicate rate and retention cleanup;
- reconciliation between domain transitions and expected outbox events.

Payload evolution also crosses retention. A relay/consumer deployed today may read an event written weeks ago. Keep compatible decoders or migrate/quarantine deliberately; do not let a rolling deploy strand old rows forever.

Checkpoint: for one event, identify database commit, relay claim, broker acknowledgement, mark-progress, consumer effect, inbox commit, offset commit, and every crash window. "At least once" becomes meaningful only when each durable position is visible.

Primary references

- Java SE 21 JDBC API, `java.sql`: https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/package-summary.html
- PostgreSQL Documentation, "Transaction Isolation": https://www.postgresql.org/docs/current/transaction-iso.html
- PostgreSQL Documentation, "Explicit Locking": https://www.postgresql.org/docs/current/explicit-locking.html
- PostgreSQL Documentation, "Deadlocks": https://www.postgresql.org/docs/current/explicit-locking.html#LOCKING-DEADLOCKS
- Flyway Documentation, "Migrations": https://documentation.red-gate.com/flyway/reference/migrations

> **Version boundary:** lock resources, isolation behavior, DDL locking, SQLState/vendor codes, and online migration techniques are database- and version-specific. Pool reset and timeout behavior is pool/driver-specific. The Java baseline is Java 21; framework conveniences do not change the database atomicity boundary.

SDE-2 CORE CHAPTER

Chapter 3 - JPA and Hibernate: Lifecycle, Fetching, Equality, and Locking

Learning objectives

After this chapter, you should be able to:

- distinguish Jakarta Persistence contracts from Hibernate features and database behavior;
- explain transient, managed, detached, and removed entity states;
- reason about the persistence context as an identity map plus unit of work;
- predict dirty checking and flush boundaries without equating flush with commit;
- choose entity equality that remains safe across generated IDs, sets, detachment, and proxies;
- detect and remove N+1 query behavior using projections, fetch joins, entity graphs, or batching;
- prevent cartesian explosions and pagination/fetch-join traps; and
- use optimistic and pessimistic locking from explicit conflict semantics.

1. Three layers of contract

Keep three authorities separate:

1. **Jakarta Persistence specification:** entity lifecycle, persistence context, query language, lock modes, and standardized API semantics;
2. **provider documentation:** Hibernate fetching options, batching properties, proxy/enhancement details, statistics, and SQL generation choices;
3. **database/driver:** SQL plan, isolation, locks, constraints, timeout enforcement, and actual concurrency.

An annotation is not a query plan. `FetchType.LAZY` expresses a contract/hint boundary defined by the spec, while the provider decides mechanisms. A generated SQL query is not guaranteed across provider versions. A pessimistic lock request maps through the provider to capabilities of the target database.

Recognition rule: in every ORM interview answer, state which layer owns the claimed behavior.

2. Entity lifecycle and persistence context

State model

TEXT

```
new/transient --persist--> managed --remove--> removed
                           |
                           +--clear/close/detach--> detached

detached --merge--> copy becomes managed (argument remains detached)
```

- **new/transient:** Java object has no managed persistence identity in the context;
- **managed:** associated with an active persistence context; changes may be synchronized;
- **detached:** retains values/identity but is no longer tracked by that context;
- **removed:** scheduled for deletion when synchronized according to transaction rules.

The persistence context normally provides identity: within one context, finding the same entity identity yields the same managed Java object under the API contract. It is also a unit of work: it tracks managed state and synchronizes changes at flush.

Do not share an `EntityManager` across arbitrary threads. Scope and thread-safety follow the specification/container integration. Entities returned beyond the transaction become detached; lazy association access may then fail or trigger an architectural dependency on an "open session in view" policy.

Concrete entity - dependency-requiring

JAVA

```java
@Entity
@Table(name = "customer_order",
       uniqueConstraints = @UniqueConstraint(
           name = "uk_order_tenant_external",
           columnNames = {"tenant_id", "external_order_id"}))
public class OrderEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private long tenantId;

    @Column(name = "external_order_id", nullable = false, length = 80)
    private String externalOrderId;

    @Version
    private long version;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private final List<OrderLineEntity> lines = new ArrayList<>();

    protected OrderEntity() { }

    public void addLine(OrderLineEntity line) {
        Objects.requireNonNull(line);
        lines.add(line);
        line.attachTo(this);
    }
}
```

The database constraint is authoritative for concurrent uniqueness. Bidirectional helper methods maintain both in-memory sides. Cascade and orphan removal encode lifecycle ownership; do not apply `CascadeType.ALL` mechanically to shared relationships such as products or users.

3. Dirty checking, flush, and commit

Formal model

For a managed entity, a provider tracks state and detects changes. At a flush, pending entity transitions are translated to SQL in an order that satisfies provider/database constraints as far as possible. Flush can occur:

- explicitly through `flush()`;
- before transaction commit;
- before certain queries depending on flush mode and affected state;
- at provider-defined/standardized synchronization points.

Flush sends/synchronizes changes to the database transaction. It does **not** mean the transaction committed. Later rollback can undo database changes. Conversely, a constraint failure may appear at flush/commit rather than at the setter call.

Execution walkthrough

JAVA

```java
@Transactional
public void rename(long tenantId, long id, String newName) {
    OrderEntity order = repository.require(tenantId, id); // managed
    order.rename(newName);                                // no explicit save needed
    auditRepository.add(...);                             // managed/persisted work
} // provider flushes; transaction manager commits or rolls back
```

If `rename` violates a length/check/unique constraint, failure can surface when SQL executes. An intermediate query may trigger flush earlier than the final method return. Do not catch a persistence exception and continue using the same transaction as if it were healthy; it may be marked rollback-only and the context can contain state that no longer represents database outcome.

Bulk DML trap

JPQL/SQL bulk updates operate directly against database rows and do not behave like per-entity mutation with ordinary managed-state synchronization and callbacks. Existing managed entities can become stale.

JAVA

```java
int changed = entityManager.createQuery("""
    update OrderEntity o
       set o.status = :newStatus
     where o.status = :oldStatus
    """)
    .setParameter("newStatus", CANCELLED)
    .setParameter("oldStatus", EXPIRED)
    .executeUpdate();
entityManager.clear(); // one deliberate policy after bulk work
```

The correct clear/refresh strategy depends on whether pending changes exist and what must remain managed. Flush intentionally before bulk DML if required; never clear away unsynchronized work accidentally.

4. persist, merge, and detached updates

`persist` makes a new entity managed under the context. `merge` copies state from its argument into a managed instance and returns that managed instance; the supplied object does not become managed merely because it was passed to `merge`.

JAVA

```java
OrderEntity managed = entityManager.merge(detached);
assert managed != detached;
```

Blindly merging a web-bound detached entity is dangerous:

- missing fields may overwrite current values;
- stale associations can cascade;
- unauthorized fields can be changed;

- a concurrent update can be lost without a version check;
- the graph can cause unexpected loads/writes.

Prefer a command-specific transaction: load the authorized managed aggregate, call invariant-protecting methods with allowed fields, and rely on version/constraint guards. DTOs do not need entity annotations.

5. Entity equality and hashing

The generated-ID problem

A new entity may have `id == null`; after persistence it receives an ID. If `hashCode()` changes while the object is in a `HashSet`, the set may no longer find it. If all transient instances with null ID are considered equal, distinct new entities collapse.

Equality options:

- immutable, assigned, globally/tenant-unique ID available at construction;
- immutable natural key whose meaning never changes;
- provider-aware generated-ID strategy documented and tested across transient/managed/detached/proxy states;
- identity equality for entities while value objects use value equality.

There is no universal five-line recipe independent of ID strategy, inheritance, proxies, and collection use. Requirements:

1. reflexive, symmetric, transitive, consistent;
2. equal objects have equal stable hash codes while stored in hash collections;
3. proxy/subclass comparisons do not break symmetry;
4. two distinct transient entities are not accidentally equal;
5. the chosen business key is immutable.

Records are excellent DTO/value types but usually a poor direct fit for mutable ORM entities with provider construction/proxy requirements. Keep value semantics where they belong.

Interview example

If the service assigns a UUID order ID before persistence, equality can use that immutable ID from construction. If the database assigns a numeric identity, avoid putting a transient entity in a hash collection whose membership must survive ID assignment, or adopt a provider-recommended strategy and test lifecycle states. State the tradeoff instead of reciting code.

6. Fetching and the N+1 problem

What N+1 means

Code loads `N` parent rows with one query, then accesses one lazily obtained association for each parent and triggers up to `N` more queries:

JAVA

```java
List<OrderEntity> orders = repository.findRecent(tenantId); // 1 query
for (OrderEntity order : orders) {
    render(order.getCustomer().getName());                  // up to N
}
```

N+1 is an *execution-shape* problem, not simply a lazy/eager annotation problem. Making every relationship eager can create huge joins, extra secondary selects, circular graph loads, and unpredictable query cost across use cases.

Recognition signals:

- query count grows with result count;
- latency rises even though each query is individually fast;
- ORM statistics/log trace shows repeated SQL differing only by ID;
- serialization walks associations outside deliberate fetch planning;
- a repository returns entities for a read projection.

Fetch-plan toolbox

Tool	Best fit	Caveats
DTO/projection query	read API needs a fixed small shape	separate mapping/query; avoids managed graph
fetch join	one use case needs association in same query	duplicates/cartesian multiplication; collection pagination traps
entity graph	select associations per query through standardized graph concepts	provider SQL still needs inspection
batch fetching	many lazy references/collections can be fetched in groups	provider feature/config; reduces, does not necessarily eliminate queries
explicit secondary query	load parent IDs/page, then related rows in bounded query	assemble carefully; often robust for paginated collections

Cartesian explosion

Fetching two to-many collections in one join can multiply rows. An order with 10 lines and 5 tags can produce 50 result rows before ORM deduplication. Network, memory, sorting, and hydration costs remain even if Java returns one order object. Multiple bag/list fetches may have provider-specific restrictions.

Decision rule: fetch exactly the graph required by one use case. For read endpoints, projection often wins. For aggregate mutation, load the needed managed state in a bounded way. Assert query count or shape in tests for critical paths.

Pagination plus collection fetch

Applying SQL limit to joined parent/child rows can cut through a parent's children or yield fewer distinct parents than requested. Providers may paginate in memory or reject/warn depending on query/version. A robust two-phase pattern is:

1. fetch one page of ordered parent IDs using a keyset query;
2. fetch required parents/children for those IDs;
3. reconstruct the requested parent order;
4. keep bounds explicit.

Do not combine collection fetch join and pageable abstraction without inspecting generated SQL and provider behavior.

7. JDBC batching and ORM batching

Batching reduces network round trips by grouping statements, but it does not change algorithmic row count or make one giant transaction safe. Provider ability to batch inserts can depend on ID generation, statement ordering, and driver/database support. Exact settings are Hibernate-specific.

For a large import:

JAVA

```java
for (int i = 0; i < commands.size(); i++) {
    entityManager.persist(toEntity(commands.get(i)));
    if ((i + 1) % batchSize == 0) {
        entityManager.flush();
        entityManager.clear();
    }
}
```

This bounds persistence-context memory, but clearing detaches entities and changes later behavior. A huge outer transaction still holds locks/log state; chunk transaction boundaries may be appropriate if the business operation supports resumability and partial progress. Batch failures can report update counts in driver-specific ways. Test exact infrastructure.

8. Optimistic and pessimistic locking in JPA

Optimistic version

`@Version` lets the provider include version checks in updates. Two contexts load version 7; one commits version 8; the second update affects no matching version and results in an optimistic-lock failure. The application must choose:

- surface a conflict to a user;
- reload and ask the user to reconcile;
- recompute and retry a commutative/internal operation;
- abandon under a deadline.

Do not catch and continue in the same failed transaction. A version on Order does not automatically protect an invariant across Inventory rows unless those rows/conflicts participate.

Pessimistic lock

Pessimistic lock modes request database locking through the provider. Actual lock strength, range, wait/timeout, follow-on locking, and unsupported combinations depend on provider/database. Lock rows in deterministic order, keep transactions short, and classify timeout/deadlock separately from "not found."

Use pessimistic locking when conflict is frequent and proceeding concurrently would waste expensive work, or when the invariant requires examining locked current state. Do not use it to mask an unbounded transaction or remote call.

TRY IT | 9. Interview questions and model checkpoints

Q1. Is a managed entity saved only when save() is called?

Model checkpoint: managed changes are tracked and synchronized at flush under the persistence context/transaction. Repository `save` conventions are framework-specific. Flush is not commit.

Q2. What causes N+1, and how do you fix it?

Model checkpoint: accessing associations across N results triggers repeated queries. Choose a use-case fetch plan-projection, fetch join, entity graph, batch, or explicit second query-and verify generated SQL/query count. Global eager fetching is not a general fix.

Q3. Why is merge risky for HTTP updates?

Model checkpoint: it copies detached state into a managed instance, potentially including stale/unauthorized fields and cascades. Load authorized current state and apply a command with version checks.

Q4. When does flush happen?

Model checkpoint: explicit flush, transaction completion, and query-related synchronization according to flush mode/spec/provider. It executes SQL inside the transaction but does not commit.

SDE-2 follow-ups

1. Design equality for a generated-ID entity used in a `Set`, including transient and proxy cases.
2. A page of 20 orders creates 421 queries. Produce a measurement-first remediation plan.
3. A fetch join returns 10,000 rows for 100 parents. Explain row multiplication and alternatives.
4. Compare `@Version` with an HTTP `If-Match` contract and show how they connect.

TRY IT | 10. Exercises

1. Draw lifecycle state transitions for `persist`, `find`, `detach`, `merge`, `remove`, clear, close, rollback, and commit.
2. Write a test that demonstrates `merge` returns a different managed instance.
3. Build a two-phase paginated fetch for parents and children; preserve parent order and bound query count.
4. Run a batch insert with two ID strategies and compare statements, memory, and transaction time.
5. Reproduce an optimistic conflict with two independent transactions and classify the API response.

RECAP | 11. Summary checklist

- [ ] Specification, provider, and database guarantees are labeled separately.
- [ ] Entity state and context lifetime are explicit.
- [ ] Flush is not confused with commit.
- [ ] DTO updates are mapped onto authorized managed state.
- [ ] Equality remains valid across lifecycle and hash-collection use.
- [ ] Every query has a deliberate fetch plan.
- [ ] Collection joins account for row multiplication and pagination.
- [ ] Batching is measured and persistence-context memory is bounded.
- [ ] Locking strategy matches invariant scope and conflict rate.

12. Association ownership and read-model laboratory

Owning side is a mapping concept

For a bidirectional association, JPA designates the side whose mapping controls the database relationship update. This does not automatically mean domain aggregate ownership. Keep both concepts explicit.

If `OrderLine` holds the foreign key, a common mapping makes the line's `@ManyToOne` the owning mapping side and `Order.lines` uses `mappedBy`. Application helper methods update both Java references:

JAVA

```java
void addLine(OrderLineEntity line) {
    if (line.order() != null) {
        throw new IllegalArgumentException("line already belongs to an order");
    }
    lines.add(line);
    line.attachTo(this);
}
```

Changing only the inverse collection can leave the generated foreign-key update absent or in-memory graph inconsistent. Test SQL and reloaded state, not only the collection before flush.

Cascade and orphan removal decision rule

Cascade answers which entity operation propagates across a relationship. Orphan removal expresses lifecycle ownership when removal from the relationship should delete the dependent entity. Use them when the child truly cannot exist independently.

An `OrderLine` commonly belongs to one Order and may be orphan-removed. A `Product` is shared; cascading remove from an order to product would be catastrophic. Many-to-many plus broad cascade is a red flag because ownership is ambiguous. Model the join as its own entity when it has attributes/lifecycle.

Large collections

An aggregate method that loads 100,000 children to add one member is not bounded. Options:

- impose aggregate size invariant;
- use a targeted database command with constraints/versioning;
- split aggregate boundaries;
- represent a child repository/query separately;
- use bulk DML under explicit context-clear rules.

Do not expose a lazy collection as a JSON page. Pagination is a query contract, not a property getter.

Read model versus managed graph

For `GET /orders/{id}/summary`, a projection query can return:

```java
record OrderSummary(long id, String status, long totalCents,
                    String customerDisplayName, long lineCount) {}
```

It avoids loading a mutable entity graph and makes selected columns/query cost explicit. A projection is not automatically fast: joins and aggregation still require an index/plan. But it reduces accidental lazy loads and serialization cycles.

Use managed entities when a transaction needs invariant-protecting state changes. Use projections for stable bounded reads. Do not update a projection and expect dirty checking.

Fetch-plan test matrix

Use case	Expected plan	Assertion
list 20 order summaries	one bounded projection query	query count constant; selected fields only
edit one order note	entity by tenant+id/version	one select plus one versioned update
display order with 20 lines	fetch join or bounded two-query plan	no N+1; row count bounded
page orders with lines	page IDs then fetch children	exactly page-size parents; stable order
export 1M orders	streaming/chunked projection under job protocol	context/memory bounded; resumable

SQL logging in tests can be noisy and brittle; provider statistics or a datasource proxy can count statements. Still inspect representative SQL and plans. A test that asserts exactly one provider-internal query forever may resist legitimate upgrades; assert the performance contract at the right granularity.

Laboratory checkpoint

When given an ORM performance problem, state context lifetime, entity state, access path, association cardinality, generated query count, rows returned, transaction/flush point, and database plan. Changing `LAZY` to `EAGER` without that model is guesswork.

Serialization-boundary checkpoint

Never hand a managed entity directly to a general JSON serializer and assume the output is stable. Accessor traversal can initialize lazy associations, create N+1 queries, follow cycles, expose internal fields, or fail after the context closes. Provider proxy types and enhancement details can also leak into behavior.

Map inside a deliberate read transaction to a bounded DTO/projection. Decide whether absent association means `null`, omitted field, empty collection, or inaccessible resource. Put a maximum on nested collection size or expose it as its own paginated resource. Test SQL count and serialized contract together.

An "open persistence context in view" policy can keep lazy loading available through rendering, but it broadens the query/connection timing boundary and makes controller/serializer access generate SQL. If selected, document and instrument it; it does not replace fetch planning. Many services disable or avoid reliance on it so repository/query code owns data access visibly.

Model answer: entities model persistence lifecycle; API DTOs model compatibility and trust. Explicit mapping is not useless boilerplate-it is the place where authorization, projection, version, units, nullability, and evolution become reviewable.

Primary references

- Jakarta Persistence specification and API: https://jakarta.ee/specifications/persistence/
- Hibernate ORM User Guide: https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html
- Spring Data JPA Reference: https://docs.spring.io/spring-data/jpa/reference/

> **Version boundary:** Jakarta Persistence contracts are distinct from Hibernate extensions. Fetch batching, proxy/enhancement behavior, generated SQL, supported hints, ID batching, and pagination safeguards vary by provider release. Select documentation that matches the managed dependency version. Examples assume Java 21 and Jakarta namespaces.

SDE-2 CORE CHAPTER

Chapter 4 - Redis and Cache Consistency: Patterns, Failures, and a Worked Case

Learning objectives

After this chapter, you should be able to:

- decide whether a cache is justified by measured latency/load and a clear source of truth;
- specify cache-aside reads and invalidation with explicit stale-data windows;
- choose TTLs from freshness, recovery, privacy, and load requirements;
- mitigate stampedes, hot keys, penetration, eviction, and failover without treating Redis as magic shared memory;
- distinguish cache consistency from database transaction consistency;
- design a cache key/version/serialization contract; and
- explain a complete read/write path with query plan, ORM fetch plan, cache failure policy, metrics, and recovery.

1. A cache is a replicated derived view

Intuition and formal model

Let database state be authoritative `D(t)`. A cache entry `C_k(t)` is a derived copy for key `k`, created at time `t_w`, and valid under a freshness policy. Cache correctness is not "value exists"; it is:

```text
key identity + serialization/schema version + authorization scope
+ source version/freshness + invalidation/expiry behavior
```

Every cache design must answer:

- What is the source of truth?
- Is the cached value an entity, projection, computation, negative result, or token?
- How stale may it be, and for which operations?
- Who writes, invalidates, expires, or rebuilds it?
- What happens when Redis is slow, unavailable, partitioned, or evicts the key?
- Can a stale value violate authorization, money, inventory, or another invariant?

- How are key and value schemas versioned?
- How is sensitive data minimized and erased?

If stale data can cause an invalid state transition, re-check the invariant in the authoritative database transaction. A cache can speed discovery but is rarely the final concurrent guard.

Recognition and decision rules

Cache when repeated reads or computations dominate, values are reusable across requests, and a defined staleness window is acceptable. Do not cache automatically when:

- hit rate will be low due to unique keys;
- data changes as often as it is read;
- correctness requires the latest value and authoritative lookup is already fast;
- payloads are huge or per-user and create memory/cardinality pressure;
- the team lacks invalidation, observability, and incident ownership;
- the database query is slow only because an obvious index/fetch problem remains.

Measure baseline query plans, p50/p95/p99, request rate, database load, and expected working set before adding a second stateful system.

2. Cache-aside

Read contract

TEXT

```
read(k):
  v = cache.get(versioned(k))
  if valid v exists: return v
  v = database.read(k)
  cache.set(versioned(k), serialize(v), ttl+jitter)
  return v
```

The application owns population. Cache failure normally falls back to the database only if that fallback is capacity-safe and within the deadline. A mass cache outage can overwhelm the source; use concurrency limits, degraded behavior, and controlled recovery.

Write/invalidation order

A common cache-aside mutation is:

1. commit database change;
2. invalidate the cache key after commit.

Why not invalidate before commit? A concurrent reader can miss, load old committed database state, repopulate it, then the writer commits-leaving stale cache. Why not write cache first? Database failure leaves uncommitted future state exposed.

Even DB-then-delete has a race:

TEXT

```
R misses cache and reads old DB value
W commits new DB value
W deletes cache
R writes old value into cache after deletion
```

TTL bounds the stale duration but does not eliminate it. Stronger options include:

- value carries a monotonic source version and writes use compare/version policy;
- short TTL plus explicit acceptable stale window;
- transaction outbox/change stream invalidates after commit, with duplicates tolerated;
- write-through or coordinated service ownership with carefully specified failure semantics;
- never cache correctness-critical transition reads; query authoritative state;
- namespace/key version bump for coarse invalidation.

There is no general atomic transaction across an ordinary relational database and Redis. Lua scripts can be atomic *inside Redis* under documented server semantics, not across systems.

Concrete Java sketch

JAVA

```java
public ProductView get(ProductId id) {
    String key = "product:v3:" + id.value();
    try {
        ProductView cached = codec.decode(cache.get(key));
        if (cached != null) {
            return cached;
        }
    } catch (CacheUnavailable failure) {
        metrics.cacheFailure("read");
    }

    ProductView loaded = repository.requireProjection(id);
    try {
        cache.set(key, codec.encode(loaded), ttlWithJitter(id));
    } catch (CacheUnavailable failure) {
        metrics.cacheFailure("fill");
    }
    return loaded;
}
```

This dependency-requiring pseudocode omits a stampede guard and timeout budget. Cache calls need tight deadlines. Serialization failures should not be treated as a normal miss forever; they indicate schema/corruption issues and need bounded eviction/alerting.

3. TTL, eviction, and serialization

TTL decision model

TTL is a correctness/recovery/load control, not merely a memory setting. Consider:

- maximum acceptable staleness;
- update frequency and invalidation reliability;
- source capacity during refill;
- working-set size and eviction policy;
- incident recovery after a bad value or missed invalidation;
- privacy/retention obligations;
- synchronized expiry and stampede risk.

Add bounded jitter so many related entries do not expire simultaneously:

```text
effectiveTTL = baseTTL * random(0.9, 1.1)
```

Jitter does not fix a hot single key. It distributes population expiry across many keys.

Key and value schema

Good key design contains a namespace and schema version, stable canonical identity, and tenant boundary if values differ by tenant:

```text
catalog-product:v3:tenant-17:sku-BOOK-21
```

Avoid secrets, raw personal data, enormous user-controlled fragments, and ambiguous concatenation. Hashing hides length/content but does not remove authorization or deletion requirements.

Use a versioned serialization schema with:

- explicit field meanings, units, nullability, and enum evolution;
- size limit and compression policy;
- backward/forward compatibility window during deploys;
- safe deserialization into DTOs, not arbitrary executable object graphs;
- metrics for decode failures and old schema hits.

An application deploy where old and new versions share Redis is a mixed-version protocol. Prefix/version keys or support both encodings during the transition.

Eviction is not expiration

Expiration removes a key after a time policy. Eviction removes keys under memory policy/pressure. Neither is exact at every instant under every deployment mode. The application must tolerate a miss at any time. If losing a key breaks correctness, that key is durable state, not a cache.

4. Stampedes, hot keys, and penetration

Stampede

If a popular key expires, thousands of requests may miss and query the database. Mitigations:

- **single-flight/request coalescing:** one loader per key in a process; other callers await under a deadline;
- **distributed lease/lock:** one loader across instances, with lease/fencing and failure recovery;
- **stale-while-revalidate:** serve bounded stale data while one refreshes;
- **probabilistic early refresh:** refresh before expiry with probability increasing near deadline;
- **prewarming:** populate known hot keys carefully before traffic shift;
- **source bulkhead:** cap concurrent fallback queries.

A distributed lock that expires while the loader is still working can allow a second loader. For pure cache fill, duplicate work may be acceptable; for side effects, use fencing/idempotency. Never wait on a cache-fill lock longer than the request deadline.

Hot key

One key can saturate a Redis shard/network/CPU even with a high hit rate. Options include local near-cache with a very short TTL, replicated/read-scaled cache architecture, sharded derived representation when semantics allow, response/CDN caching, or eliminating per-request fetch through configuration snapshotting. Replicating the key raises invalidation complexity and staleness.

Cache penetration

Requests for nonexistent keys repeatedly hit the database. Negative caching stores "not found" briefly, but authorization and creation races matter. A user forbidden from seeing a record should not poison a global "not found" entry. If the object can be created soon, keep the negative TTL short or invalidate on create. Bloom filters can reject definitely absent values with false positives (never false negatives under a correctly maintained basic model), but maintaining them adds another consistency problem.

Failure modes

Failure	Symptom	Safe response
Redis timeout	latency/threads blocked	tight timeout, bulkhead, fallback or degrade under source capacity
full cache flush/restart	miss storm	staged warming, fallback limit, stale/local layer where acceptable
replica lag/failover	older cached values	version/freshness policy; authoritative recheck for writes
hot key	one shard saturated	local/CDN layer, replicate carefully, redesign access
serialization mismatch	decode failures	versioned key/codec, evict bounded bad entry, alert
missed invalidation	stale reads	TTL/version/event reconciliation; never rely on cache for invariant
oversized value	network/heap/GC latency	projections, size limit, split only with clear consistency

5. Worked case: product catalog plus price update

Requirements

- Product details are read 20,000 times/second at peak.
- Database can sustainably serve 2,000 product reads/second with headroom.
- Descriptions may be stale for 5 minutes; displayed price may be stale for at most 10 seconds.
- Checkout must never use cached price as the authoritative charge.
- Product updates occur around 50/second, heavily skewed to a small hot set.
- A rolling deploy can have two codec versions active.

Data and query design

Authoritative tables:

```sql
product(id, tenant_id, sku, description, active, version, updated_at)
product_price(product_id, currency, amount_minor, version, updated_at)
```

Unique `(tenant_id, sku)`. Read projection joins by product ID/currency and selects only API fields. An index supports tenant/SKU lookup; primary/foreign-key access supports ID. Inspect plans under skewed data.

Separate cache entries by freshness class:

```text
product-description:v2:{tenant}:{productId} TTL about 5m + jitter
product-price:v4:{tenant}:{productId}:{currency} TTL about 10s + jitter
```

This prevents an infrequently changing description from forcing price staleness and allows price invalidation independently. Values include source version and `asOf` instant. The API documents that catalog price is indicative; checkout re-reads and validates authoritative price in its transaction/workflow.

Write path

1. authorization checks product-management capability and tenant;
2. transaction loads/updates price with expected version;
3. database constraint/check protects amount and currency rules;
4. transaction inserts `ProductPriceChanged` outbox event with source version;
5. commit;
6. local best-effort cache delete may reduce common latency but is not the sole correctness mechanism;
7. outbox relay publishes versioned invalidation/update event;
8. cache consumer deletes or conditionally replaces only if event source version is newer;
9. TTL recovers from a lost invalidation;
10. reconciliation samples cache/source versions and alerts on excessive lag.

If events arrive out of order, an older change must not overwrite a newer cached version. Delete is often simpler and naturally safe under duplicate delivery, though the earlier cache-fill race remains bounded by TTL/version checks.

Read path and stampede control

1. read cache with a strict sub-deadline;
2. validate codec/schema and freshness;
3. on hit, return with `asOf` metadata where product requires transparency;
4. on miss, acquire a per-key single-flight slot;
5. winner reads bounded database projection and fills TTL+jitter;
6. waiters share result only within their deadline;
7. database fallback concurrency is capped;
8. if source is overloaded, serve a bounded stale description but fail/degrade price according to product policy.

This uses a local single-flight, so each instance may still load once. If that residual load is too high, consider a distributed lease but evaluate failure complexity. Prewarm the measured top hot products before a planned cache restart.

Consistency walkthrough

At `t0`, cached price version 7 is $10. At `t1`, admin commits version 8 at $12. Before invalidation reaches all readers, they can see version 7 for up to the policy window. Checkout ignores catalog cache, reads version 8, and quotes/charges under its own price-lock contract. If an old version-7 event arrives after version 8, conditional version logic prevents rollback. If all invalidations fail, 10-second TTL bounds the catalog display error.

The product must decide whether that is acceptable; engineering cannot call it "eventually consistent" and stop. Regulatory or contractual price display rules may require a different design.

Observability

Track by cache name/operation/outcome, never product ID:

- hit, miss, stale-served, negative hit, decode failure;
- get/set/delete latency and timeout;
- single-flight waiter count and load duration;
- fallback database concurrency/rejection;
- estimated working set, memory, eviction, expiry;
- invalidation event lag and out-of-order count;
- source/cache version sample delta;
- hot-key evidence through sampled logs/traces, not unbounded metric labels.

High hit ratio is not sufficient. A 99% hit ratio at 20,000/s still sends 200 reads/s; one hot miss wave can exceed the source. Measure tail latency and source protection.

TRY IT | 6. Interview questions and model checkpoints

Q1. Which should happen first: database write or cache delete?

Model checkpoint: usually commit authoritative DB change, then invalidate after commit. Invalidate-before-write permits old-value repopulation before commit. DB-then-delete still has a read/fill race; bound it with TTL/version/event protocol.

Q2. How do you choose TTL?

Model checkpoint: freshness tolerance, invalidation reliability, refill/source capacity, working set/eviction, privacy, bad-value recovery, and expiry synchronization. Add jitter; separate data with different freshness needs.

Q3. Can Redis make a database transaction distributed?

Model checkpoint: an atomic Redis operation is local to Redis. It does not atomically commit with a relational transaction. Use recoverable events/outbox, idempotency, or a deliberately selected distributed protocol.

Q4. What is a cache stampede?

Model checkpoint: many callers miss/expire the same reusable value and concurrently load the source. Use coalescing, early refresh, bounded stale serving, prewarm, and source bulkheads; define lock failure and deadlines.

Q5. What makes a cache key safe?

Model checkpoint: unambiguous versioned namespace, correct tenant/authorization scope, canonical identity, length bounds, no secrets/PII, and migration/deletion semantics.

SDE-2 follow-ups

1. A cache cluster fails and fallback takes the database down. Design staged recovery and admission controls.
2. A GDPR deletion removes the database row but personal data remains in cache. Add lifecycle and evidence.
3. Two application versions disagree on enum encoding. Design a mixed-version rollout.
4. Cache hit ratio is 99.8% but p99 regressed. Enumerate payload, network, hot-key, GC, and fallback hypotheses.

TRY IT | 7. Exercises

1. Draw all interleavings of cache miss/fill and database write/delete; mark stale outcomes.
2. Choose TTL and invalidation for permissions, product text, price, feature flags, and one-time tokens. Explain why one policy cannot fit all.
3. Design a version-aware invalidation consumer robust to duplicates and out-of-order events.
4. Calculate fallback load at 50,000 reads/s for hit ratios from 90% to 99.99%, then add a 30-second cold-start scenario.
5. Specify a chaos test for Redis latency, partial failover, mass eviction, and codec mismatch.

8. Final persistence-and-cache checklist

- [ ] The source of truth and permitted staleness are named per field/use case.
- [ ] Database query and ORM fetch plans are already efficient and bounded.
- [ ] Cache keys and values are scoped, versioned, size-bounded, and privacy-safe.
- [ ] DB/cache ordering and every stale race are documented.
- [ ] TTL, jitter, invalidation, and reconciliation provide layered recovery.
- [ ] Stampede/hot-key/penetration and source overload have controls.
- [ ] Checkout/state transitions re-check authoritative invariants.
- [ ] Cache outage behavior fits the request deadline and source capacity.
- [ ] Metrics cover hit quality, latency, fallback, evictions, lag, and version drift.

9. Cache operating laboratory

Pattern comparison

Pattern	Writer/read behavior	Strength	Failure burden
cache-aside	app loads on miss, invalidates after DB commit	simple, demand-populated	stale fill race, stampede, cold start
read-through	cache abstraction loads from source	central loader policy	cache becomes critical coordinator; loader failures
write-through	write passes through cache to source	cache updated on write path	source/cache atomicity and latency still need definition
write-behind	cache acknowledges before asynchronous source write	low write latency	cache is now durable workflow/state; loss/order/replay risk
refresh-ahead	refresh before expiry	reduces popular-key misses	refreshes unused data; failure/backpressure
near-cache	process-local copy in front of shared cache	hot-read latency and shared-cache relief	per-instance staleness/invalidation and memory

Names do not define guarantees. Write the commit/ack sequence and crash state. A "write-behind cache" that can lose acknowledged writes is not a cache optimization; it changes durability semantics.

Cold-start capacity drill

Peak read traffic is 30,000/s, normal hit ratio 98%, and DB headroom is 1,000 reads/s. Normal misses are 600/s. A cache flush causes up to 30,000 fallback reads/s-30x headroom.

Recovery plan:

1. cap fallback concurrency below safe DB budget;
2. shed/degrade noncritical reads or serve bounded stale near-cache;
3. coalesce hot-key loads;
4. warm measured hot set in controlled batches;
5. restore traffic gradually while watching DB latency/pool and cache fill;
6. avoid every instance prewarming the same full dataset;
7. preserve an emergency bypass/disable switch with audit;
8. test this before an incident.

High availability of Redis does not eliminate logical flush, codec deploy, mass expiry, network partition, or client timeout.

Cache correctness test matrix

- read miss -> source -> fill -> subsequent hit;
- write commit -> delete failure -> TTL/event eventually repairs;
- concurrent miss/read with write/delete interleaving -> staleness bounded;
- out-of-order invalidation versions -> old event cannot erase/overwrite newer value incorrectly;
- duplicate invalidation -> safe no-op;
- negative cache then create -> create invalidates or short TTL bounds absence;
- tenant A key cannot return tenant B value;
- corrupt/old codec -> bounded eviction and source reload, with metric;
- Redis slow -> deadline/bulkhead protects request and source;
- mass expiry -> coalescing and fallback admission hold DB below budget;
- privacy deletion -> keys and derived values removed within declared SLO.

Hot-key decision checkpoint

Start with evidence: per-shard CPU/network/ops, sampled key frequency, payload size, client connections, and request path. Local near-cache helps read hot keys but increases stale copies. Splitting a counter requires merge/error semantics. Replicating reads may move load to network or replicas. Sometimes the correct fix is to remove per-request cache lookup by publishing an immutable configuration snapshot to each process.

Laboratory checkpoint

An SDE-2 cache answer includes source capacity during total cache loss. If the fallback cannot survive, caching increased normal capacity but created a cliff; admission, staged recovery, or a different authoritative read architecture must address it.

Primary references

- Redis Documentation, "Client-side caching": https://redis.io/docs/latest/develop/clients/client-side-caching/
- Redis Documentation, "Key eviction": https://redis.io/docs/latest/develop/reference/eviction/
- Redis command documentation, `EXPIRE`: https://redis.io/docs/latest/commands/expire/
- Redis Documentation, "Distributed locks with Redis": https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/
- PostgreSQL Documentation: https://www.postgresql.org/docs/current/

> **Version boundary:** Redis expiration timing, eviction, replication/failover, cluster routing, client tracking, and scripting/function behavior depend on server/client versions and topology. Treat official deployment documentation as authoritative. The worked Java service baseline is Java 21; later JDK/framework features are optional.

SDE-2 CORE CHAPTER

Chapter 5 - Capstone: One Data Path Across Cache, ORM, SQL, and Events

This chapter joins the existing relational, transaction, JPA, and cache chapters into one observable data path. It deliberately avoids repeating the dedicated MySQL, Hibernate/JPA, Spring Data, and Redis books.

> **From Vinay:** A repository method and a cache annotation make code shorter, not the data path simpler. In an SDE-2 interview, narrate every store boundary: which version you read, which lock or predicate protects the write, what committed, and how stale copies are repaired.

1. Read path: derived cache to authoritative row

Consider `GET /products/{id}` where product descriptions tolerate 30 seconds of staleness but checkout prices do not.

```text
request + tenant/authorization
 -> build versioned, scoped cache key
 -> Redis client queue/network/GET
    -> hit: decode schema + verify logical freshness/version
    -> miss/unusable: enter per-key single-flight
       -> acquire DB pool connection
       -> JPA/SQL query with tenant predicate + projection
       -> optimizer/index/storage produces row
       -> map authoritative row to read model
       -> cache SET with TTL+jitter and source version
 -> return DTO
```

Checkout must read the authoritative price or use an explicit price-lock contract. A catalog cache is not upgraded into financial truth because its hit ratio is high.

Lower-level cost table

Convenience call	Hidden path to measure
cache `get`	serializer, client queue/connection, cluster slot/node, response bytes, decode/version
repository `find`	possible auto-flush, SQL generation, pool acquisition, server plan/I/O, entity tracking
lazy getter	proxy/collection initialization and possibly another SQL statement
DTO projection	query/result mapping without managed mutation; still database work
cache `put`	encode, network, TTL/eviction policy; write outcome may be unknown on timeout

2. Write path: one local commit, recoverable propagation

```text
PATCH /products/{id} If-Match: version-7
 -> transaction starts
 -> UPDATE product
      SET price=?, version=version+1
      WHERE tenant_id=? AND id=? AND version=7
 -> require affected rows = 1
 -> INSERT outbox(event_id, aggregate_id, version=8, payload)
 -> COMMIT product + outbox
 -> return version-8 ETag

outbox relay -> Kafka ProductChanged(v8)
cache invalidator -> delete/replace only if event version is not older
readers -> miss/refill version 8
```

The database transaction owns product and outbox. Redis and Kafka are separate atomic domains. A relay crash after publish produces a duplicate; consumers use event ID/version. A cache delete timeout leaves an unknown result; TTL and later invalidation/reconciliation repair it.

3. Flush, commit, response, and propagation are different points

```text
managed entity mutation
 -> JPA dirty checking notices change
 -> flush emits versioned SQL
 -> database accepts statements
 -> commit makes local transaction durable
 -> HTTP response may be lost
 -> outbox publication may happen later or duplicate
 -> cache invalidation may lag/fail/reorder
```

Calling repository `save`, JPA `flush`, Kafka `send`, or Redis `delete` is not one global commit. State exactly which result has been acknowledged.

4. Failure and consistency windows

Window	Observable risk	Contract/repair
cache miss loads v7; DB commits v8; old reader fills v7	stale value resurrected	source version in value; reject older fill or second invalidation
JPA flush succeeds; transaction rolls back	SQL appeared in logs but no commit	distinguish flush from durable state
DB commit succeeds; HTTP response lost	client sees failure, row exists	idempotency/conditional replay/read reconciliation
DB commit succeeds; outbox unpublished	downstream stale	durable relay backlog/age and retry
relay publishes; crashes before marking sent	duplicate event	consumer inbox/event ID
v8 invalidation arrives before delayed v7	old event erases newer cache	version-aware invalidation
Redis timeout on delete	deletion may have happened	safe repeated delete plus TTL/reconciliation
connection acquired, then waits on lock	pool slot held while no progress	lock/query deadlines and short transaction
bulk JPQL updates rows	managed context/cache stale	flush, bulk, clear/evict and version policy
replica read after source write	old row	source/causal routing for read-your-write

5. Concurrency choices across layers

Conditional SQL before distributed locks

For stock or versioned state, prefer a database atomic predicate:

SQL

```
UPDATE inventory
SET available = available - :quantity,
    version = version + 1
WHERE sku = :sku
  AND available >= :quantity;
```

One affected row means success. Redis locking does not strengthen the database if its lease expires and a paused owner writes later. Use database constraints/version/fencing at the authoritative resource.

Optimistic versus pessimistic

Optimistic versioning fits low-conflict, replayable commands. Pessimistic locks fit high contention or expensive late failure but hold pool connections/locks and can deadlock. A single conditional update can be clearer than either entity-level pattern.

Retry boundaries

Retry the entire local transaction after a classified deadlock/serialization conflict, not one failed statement in a partially attempted unit. Keep emails, HTTP calls, and non-idempotent publishes outside. Bound attempts by the caller deadline and observe amplification.

6. Capacity is a connected queueing system

```text
HTTP concurrency
 -> DB pool waiters -> active DB connections -> locks/CPU/I/O
 -> cache fallback concurrency -> additional DB demand
 -> outbox backlog -> broker/downstream demand
```

A 99% hit ratio at 100,000 reads/s still sends 1,000 DB reads/s. A cold cache sends 100,000/s unless single-flight/admission/stale serving protects the source. Increasing the connection pool can move the queue into MySQL and worsen contention.

For `N` application instances each with pool maximum `P`, budget approximately `N × P` against database connection and workload capacity, including migration jobs, relays, administration, failover headroom, and nested `REQUIRES_NEW` connections.

Virtual threads and asynchronous APIs do not expand the database's capacity. Backpressure must exist before scarce resources.

7. Evidence-first diagnosis

Trace one request through:

```text
cache outcome/latency/payload version
 -> single-flight wait or source fallback
 -> pool acquisition
 -> transaction duration
 -> normalized SQL + bind distribution
 -> query count/plan/rows examined/lock wait
 -> flush/commit
 -> outbox age/publication attempts
 -> invalidation lag/cache version drift
```

Do not use product IDs, SQL binds, or cache keys as unbounded metric labels. Put high-cardinality correlation in sampled traces/logs with redaction.

Symptom matrix

Symptom	Plausible layer	Evidence
endpoint slow but query 10 ms	pool/cache/network/serialization	span breakdown
query count grows with results	N+1	statement budget and call site
p99 rises during cache recovery	fallback cliff/stampede	miss waves, DB concurrency
stale value after update	invalidation/fill ordering	source/cache/event versions
deadlocks after batch feature	inconsistent row order/wider scan	deadlock graph and plan
memory grows in import	persistence context retains entities	heap/entity count; flush+clear cadence
outbox age grows	relay/broker/downstream bottleneck	claim/send/mark stages

8. Seven live interview chains with worked answers

Interview 1 - 99% cache hit but slow p99

Interviewer: "Redis hit ratio is 99%; why is the endpoint slow?"

Candidate: "Hit ratio says neither hit latency nor payload cost. I split client queue/network, Redis server, payload bytes, decode/GC, and fallback. At high volume the 1% misses may saturate the DB, and one hot key can dominate a shard. I compare p99 by cache outcome and source fallback rather than add capacity blindly."

Interview 2 - stale fill race

Interviewer: "A deleted cache value returns old data after an update."

Candidate: "A miss loaded v7 before the write; v8 committed and invalidated; then the slow reader filled v7. I carry source version and reject an older fill, or use versioned keys/second invalidation. TTL bounds but does not prevent the race. Checkout revalidates authoritative price."

Interview 3 - N+1 under a repository

Interviewer: "One repository method generates 501 queries."

Candidate: "The root query returned 500 managed entities and a lazy association was traversed. I capture the use-case shape and use a DTO projection, fetch join/entity graph for a bounded aggregate, or batch/two-step load. I compare query count with row multiplication and add a statement-budget test."

Interview 4 - optimistic conflict API

Interviewer: "Two admins edit version 7."

Candidate: "Both send `If-Match`/expected version. The database update includes `WHERE version=7`; one changes to 8, the other affects zero rows. I return `412 Precondition Failed` for HTTP conditional mismatch, include current representation/link as policy allows, and never silently merge fields the caller did not own."

Interview 5 - deadlock retry with outbox

Interviewer: "Order transaction deadlocks after writing outbox."

Candidate: "The victim transaction rolls back both order and outbox. I retry the whole deterministic transaction with the same request/event identity, bounded backoff+jitter, and stable row order. No external publish occurs inside the attempt; the relay sees only the committed outbox row."

Interview 6 - pool exhaustion

Interviewer: "Queries are fast, but every request waits one second."

Candidate: "I inspect pool acquisition separately. Long transactions, remote calls inside transactions, leaks, lock waits, nested `REQUIRES_NEW`, or total per-instance pools may exhaust capacity. I fix hold time/admission and global budget before raising pool size, which could only move overload into the database."

Interview 7 - zero-downtime persistence change

Interviewer: "Split `price` into amount and currency with mixed app versions."

Candidate: "Expand with nullable/compatible new fields, deploy readers/writers that support both and define authority, backfill in bounded indexed batches, verify counts/value equivalence and cache/event schema compatibility, switch reads, tighten constraints, then remove old fields later. I monitor locks, replication, outbox lag, and rollback at each stage."

TRY IT | 9. Focused exercises and solution sketches

1. **Draw the read path.** Include cache decode failure and single-flight. **Solution:** treat corrupt entry as distinct from miss, evict/quarantine once, bound source concurrency, and record outcome metrics.
2. **Prove stale-fill repair.** Interleave reader v7 and writer v8; assert `putIfNewerOrEqual` rejects v7 after v8. The existing companion models this rule.
3. **Design offset-like outbox claims.** Use stable event ID, claim token/lease, publish, then conditional mark. Crash after publish repeats safely; stale claimant cannot mark a newer lease.
4. **Budget cold-cache load.** At 40,000 reads/s and safe DB capacity 800/s, allow at most 800 unique fallbacks/s, coalesce duplicates, degrade/shed the rest, and warm measured hot keys gradually.
5. **Test transaction truth.** Use the target database to assert duplicate constraints, version conflict, deadlock retry of the whole unit, commit-time failure, and outbox atomicity. H2 alone cannot prove MySQL locks/plans.

10. Capstone boundary

Use the dedicated MySQL volume for InnoDB/SQL internals, Hibernate/JPA for lifecycle/mapping, Spring Data for repository behavior, and Redis for structures/topology. This capstone tests whether you can join those layers into one correctness and observability story.

Primary references

- Jakarta Persistence specification and Hibernate ORM User Guide.
- MySQL Reference Manual for indexes, transactions, InnoDB, and recovery.
- Spring Framework transaction/data-access references and Spring Data JPA reference.
- Redis documentation for expiration, eviction, persistence, replication, Cluster, and scripting.

SDE-2 CORE CHAPTER

Chapter 6 - Executable Persistence and Cache Model

This dependency-free Java 21 model makes optimistic locking, fenced outbox claims, version-aware cache invalidation, and stale-fill protection testable as explicit contracts.

JAVA (continued 1/12)

```java
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Dependency-free Java 21 executable models for optimistic updates, keyset
 * pagination, and a version-aware cache-aside read path.
 */
public final class PersistencePatterns {
    private PersistencePatterns() {
    }

    public record Order(String id, long tenantId, Instant createdAt,
                        String status, long version, String note) {
        public Order {
            requireText(id, "id");
            Objects.requireNonNull(createdAt, "createdAt");
            requireText(status, "status");
            note = note == null ? "" : note;
            if (version < 0) {
                throw new IllegalArgumentException("version must be nonnegative");
            }
        }

        Order withNote(String newNote) {
```

JAVA (continued 2/12)

```
            return new Order(id, tenantId, createdAt, status,
                    Math.addExact(version, 1), newNote);
        }
    }

    public record Cursor(Instant createdAt, String id) {
        public Cursor {
            Objects.requireNonNull(createdAt, "createdAt");
            requireText(id, "id");
        }
    }

    public sealed interface UpdateResult
            permits Updated, Conflict, NotFound {
    }

    public record Updated(Order order) implements UpdateResult {
    }

    public record Conflict(long currentVersion) implements UpdateResult {
    }

    public record NotFound() implements UpdateResult {
    }

    /** In-memory stand-in for one atomic database UPDATE predicate. */
    public static final class OptimisticOrderStore {
        private final Map<String, Order> rows = new HashMap<>();

        public synchronized void insert(Order order) {
            Objects.requireNonNull(order, "order");
            if (rows.putIfAbsent(order.id(), order) != null) {
                throw new IllegalStateException("duplicate order id");
            }
```

JAVA (continued 3/12)

```
        }

        public synchronized Optional<Order> find(long tenantId, String id) {
            Order order = rows.get(id);
            return order != null && order.tenantId() == tenantId
                    ? Optional.of(order) : Optional.empty();
        }

        public synchronized UpdateResult updateNote(
                long tenantId, String id, long expectedVersion, String note) {
            if (note == null || note.length() > 200) {
                throw new IllegalArgumentException("invalid note");
            }
            Order current = rows.get(id);
            if (current == null || current.tenantId() != tenantId) {
                return new NotFound();
            }
            if (current.version() != expectedVersion) {
                return new Conflict(current.version());
            }
            Order changed = current.withNote(note);
            rows.put(id, changed);
            return new Updated(changed);
        }

        public synchronized List<Order> page(
                long tenantId, Cursor after, int limit) {
            if (limit < 1 || limit > 100) {
                throw new IllegalArgumentException("limit must be 1..100");
            }
            Comparator<Order> newestFirst = Comparator
                    .comparing(Order::createdAt).reversed()
                    .thenComparing(Order::id, Comparator.reverseOrder());
            return rows.values().stream()
```

JAVA (continued 4/12)

```
                    .filter(order -> order.tenantId() == tenantId)
                    .filter(order -> after == null
                            || order.createdAt().isBefore(after.createdAt())
                            || (order.createdAt().equals(after.createdAt())
                                && order.id().compareTo(after.id()) < 0))
                    .sorted(newestFirst)
                    .limit(limit)
                    .toList();
        }
    }

    public record ProductView(String id, long sourceVersion,
                              String description, long priceMinor) {
        public ProductView {
            requireText(id, "id");
            requireText(description, "description");
            if (sourceVersion < 0 || priceMinor < 0) {
                throw new IllegalArgumentException("negative version/price");
            }
        }
    }

    public interface ProductSource {
        ProductView require(String id);
    }

    private record CacheEntry(ProductView value, Instant expiresAt) {
    }

    /**
     * One-process cache model. Production Redis calls require strict deadlines,
     * serialization versions, source protection, and distributed failure policy.
     */
    public static final class VersionAwareCache {
```

JAVA (continued 5/12)

```
        private final Map<String, CacheEntry> values = new HashMap<>();
        private final Clock clock;

        public VersionAwareCache(Clock clock) {
            this.clock = Objects.requireNonNull(clock, "clock");
        }

        public synchronized Optional<ProductView> get(String key) {
            CacheEntry entry = values.get(key);
            if (entry == null) {
                return Optional.empty();
            }
            if (!entry.expiresAt().isAfter(clock.instant())) {
                values.remove(key);
                return Optional.empty();
            }
            return Optional.of(entry.value());
        }

        /** An older invalidation must not remove a newer filled value. */
        public synchronized boolean invalidateIfNotNewer(
                String key, long sourceVersion) {
            CacheEntry current = values.get(key);
            if (current == null
                    || current.value().sourceVersion() <= sourceVersion) {
                values.remove(key);
                return current != null;
            }
            return false;
        }

        /** An out-of-order older fill must not replace a newer value. */
        public synchronized boolean putIfNewerOrEqual(
                String key, ProductView value, Duration ttl) {
```

JAVA (continued 6/12)

```
            Objects.requireNonNull(value, "value");
            requirePositive(ttl, "ttl");
            CacheEntry current = values.get(key);
            if (current != null
                    && current.value().sourceVersion() > value.sourceVersion()) {
                return false;
            }
            values.put(key, new CacheEntry(value, clock.instant().plus(ttl)));
            return true;
        }
    }

    public static final class CachedProducts {
        private final ProductSource source;
        private final VersionAwareCache cache;
        private final Duration ttl;

        public CachedProducts(
                ProductSource source, VersionAwareCache cache, Duration ttl) {
            this.source = Objects.requireNonNull(source, "source");
            this.cache = Objects.requireNonNull(cache, "cache");
            this.ttl = requirePositive(ttl, "ttl");
        }

        public ProductView get(String id) {
            String key = cacheKey(id);
            Optional<ProductView> hit = cache.get(key);
            if (hit.isPresent()) {
                return hit.get();
            }
            ProductView loaded = source.require(id);
            cache.putIfNewerOrEqual(key, loaded, ttl);
            return cache.get(key).orElse(loaded);
        }
```

JAVA (continued 7/12)

```
        public void onProductChanged(String id, long sourceVersion) {
            cache.invalidateIfNotNewer(cacheKey(id), sourceVersion);
        }

        private static String cacheKey(String id) {
            requireText(id, "id");
            return "product:v1:" + id;
        }
    }

    public record OutboxClaim(String eventId, long claimToken) {
        public OutboxClaim {
            requireText(eventId, "eventId");
            if (claimToken <= 0) {
                throw new IllegalArgumentException(
                        "claimToken must be positive");
            }
        }
    }

    private enum OutboxState { NEW, CLAIMED, PUBLISHED }

    private static final class OutboxEntry {
        private final long aggregateVersion;
        private OutboxState state = OutboxState.NEW;
        private long claimToken;

        private OutboxEntry(long aggregateVersion) {
            this.aggregateVersion = aggregateVersion;
        }
    }

    /**
```

JAVA (continued 8/12)

```
     * In-memory model of durable outbox claim fencing. A stale relay attempt
     * cannot mark a row published after another attempt reclaims it.
     */
    public static final class OutboxStore {
        private final Map<String, OutboxEntry> events = new HashMap<>();
        private long nextClaimToken = 1;

        public synchronized boolean insert(
                String eventId, long aggregateVersion) {
            requireText(eventId, "eventId");
            if (aggregateVersion < 0) {
                throw new IllegalArgumentException(
                        "aggregateVersion must be nonnegative");
            }
            return events.putIfAbsent(
                    eventId, new OutboxEntry(aggregateVersion)) == null;
        }

        public synchronized Optional<OutboxClaim> claim(String eventId) {
            OutboxEntry entry = events.get(eventId);
            if (entry == null || entry.state != OutboxState.NEW) {
                return Optional.empty();
            }
            entry.state = OutboxState.CLAIMED;
            entry.claimToken = nextClaimToken++;
            return Optional.of(new OutboxClaim(eventId, entry.claimToken));
        }

        public synchronized boolean release(OutboxClaim claim) {
            OutboxEntry entry = events.get(claim.eventId());
            if (!owns(entry, claim)) {
                return false;
            }
            entry.state = OutboxState.NEW;
```

JAVA (continued 9/12)

```
            return true;
        }

        public synchronized boolean markPublished(OutboxClaim claim) {
            OutboxEntry entry = events.get(claim.eventId());
            if (!owns(entry, claim)) {
                return false;
            }
            entry.state = OutboxState.PUBLISHED;
            return true;
        }

        public synchronized boolean isPublished(String eventId) {
            OutboxEntry entry = events.get(eventId);
            return entry != null && entry.state == OutboxState.PUBLISHED;
        }

        public synchronized long aggregateVersion(String eventId) {
            OutboxEntry entry = events.get(eventId);
            if (entry == null) {
                throw new IllegalArgumentException("unknown event");
            }
            return entry.aggregateVersion;
        }

        private static boolean owns(
                OutboxEntry entry, OutboxClaim claim) {
            return entry != null
                    && entry.state == OutboxState.CLAIMED
                    && entry.claimToken == claim.claimToken();
        }
    }

    public static void main(String[] args) {
```

JAVA (continued 10/12)

```
        optimisticUpdateAssertions();
        keysetAssertions();
        cacheAssertions();
        outboxAssertions();
        System.out.println("PersistencePatterns assertions passed");
    }

    private static void optimisticUpdateAssertions() {
        var store = new OptimisticOrderStore();
        Instant now = Instant.parse("2026-01-01T00:00:00Z");
        store.insert(new Order("O-1", 7, now, "PENDING", 0, ""));

        UpdateResult first = store.updateNote(7, "O-1", 0, "door");
        assert first instanceof Updated;
        assert ((Updated) first).order().version() == 1;

        UpdateResult stale = store.updateNote(7, "O-1", 0, "stale");
        assert stale.equals(new Conflict(1));
        assert store.updateNote(8, "O-1", 1, "cross-tenant")
                instanceof NotFound;
    }

    private static void keysetAssertions() {
        var store = new OptimisticOrderStore();
        Instant t = Instant.parse("2026-01-01T10:00:00Z");
        Order newest = new Order("O-3", 7, t, "NEW", 0, "");
        Order tieSecond = new Order("O-2", 7, t, "NEW", 0, "");
        Order older = new Order("O-1", 7, t.minusSeconds(1), "NEW", 0, "");
        store.insert(older);
        store.insert(tieSecond);
        store.insert(newest);
        store.insert(new Order("OTHER", 8, t.plusSeconds(3), "NEW", 0, ""));

        assert store.page(7, null, 2).equals(List.of(newest, tieSecond));
```

JAVA (continued 11/12)

```
        assert store.page(7, new Cursor(t, "O-2"), 2)
                .equals(List.of(older));
    }

    private static void cacheAssertions() {
        Clock fixed = Clock.fixed(
                Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
        var cache = new VersionAwareCache(fixed);
        var calls = new AtomicInteger();
        ProductView version8 = new ProductView("P-1", 8, "Book", 1_200);
        ProductSource source = id -> {
            calls.incrementAndGet();
            assert id.equals("P-1");
            return version8;
        };
        var products = new CachedProducts(
                source, cache, Duration.ofSeconds(10));

        assert products.get("P-1").equals(version8);
        assert products.get("P-1").equals(version8);
        assert calls.get() == 1;

        products.onProductChanged("P-1", 7); // older event cannot remove v8
        assert products.get("P-1").equals(version8);
        assert calls.get() == 1;

        products.onProductChanged("P-1", 8);
        assert products.get("P-1").equals(version8);
        assert calls.get() == 2;

        boolean acceptedOlder = cache.putIfNewerOrEqual(
                "product:v1:P-1",
                new ProductView("P-1", 7, "Old", 1_000),
                Duration.ofSeconds(10));
```

JAVA (continued 12/12)

```
        assert !acceptedOlder;
    }

    private static void outboxAssertions() {
        var outbox = new OutboxStore();
        assert outbox.insert("event-8", 8);
        assert !outbox.insert("event-8", 8);
        assert outbox.aggregateVersion("event-8") == 8;

        OutboxClaim first = outbox.claim("event-8").orElseThrow();
        assert outbox.claim("event-8").isEmpty();
        assert outbox.release(first);

        OutboxClaim second = outbox.claim("event-8").orElseThrow();
        assert !outbox.markPublished(first);
        assert outbox.markPublished(second);
        assert outbox.isPublished("event-8");
    }

    private static void requireText(String value, String name) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(name + " must not be blank");
        }
    }

    private static Duration requirePositive(Duration value, String name) {
        Objects.requireNonNull(value, name);
        if (value.isZero() || value.isNegative()) {
            throw new IllegalArgumentException(name + " must be positive");
        }
        return value;
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: schema, query, index, and transaction models
- Interview Core: isolation, JPA lifecycle, N+1, and locking
- SDE-2 Follow-up: migrations, pools, outbox, and cache consistency
- Challenge: defend the worked product persistence path

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: FW 08 - Redis for Java Backend Interviews

Series index

Next book: FW 10 - Apache Kafka and Spring Kafka

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 09 of 12 - Study Step FW-09. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.